Seminar 5

Lazy evaluation. Programming with streams

The purpose of this labwork is to familiarise you with lazy evaluation, and to practice programming with streams.

1 Lazy evaluation

The evaluation of an expression is **lazy** if it is delayed until it is really needed. We can "freeze" the evaluation of an expression e by placing it in the body of a nullary function:

```
(define frozen-e (lambda () e))
```

and evaluate it anytime later by calling

```
(frozen-e)
```

For example:

```
> (define fexpr (lambda () (+ 1 2)))  ;  freeze the evaluation of (+ 1 2)
> fexpr                               ;  fexpr is a nullary function
#<procedure:fexpr>
> (fexpr)                             ; enforce the evaluation by calling the nullary function
3
```

We can also define *lazy functions* by wrapping their body in `(lambda () ...)`. For example:

```
> (define (lazy-sum x y) (lambda () (+ x y)))
> (lazy sum 1 2)
#<procedure>
> ((lazy-sum 1 2))
3
```

Remark: `(sum 1 2)` returns a function that expects 0 input arguments (that is, a nullary function). A nullary function call of `f` is `(f)`.

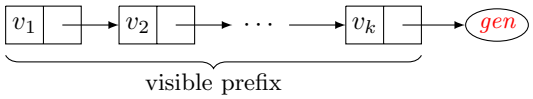
1.1 Application of lazy evaluation: Streams

We call *stream* is a finite representation of an infinite list `'(`v_1 v_2 v_3 `...)`. The general structure of a stream is

gen or `'(`v_1 ... v_k `.` *gen*`)`

A stream for an infinite list `'(`v_1 v_2 v_3 ...`)` consists of two parts:

1. A visible prefix `'(`v_1 ... v_k`)` of the infinite list.
2. A nullary function *gen*, called **generator**, which can be called to generate lazily the rest of the list. More precisely, *gen* must be defined such that the function call `(`*gen*`)` returns `'(`v_{k+1} ... v_{k+m} `.` gen'`)` where $m \geq 1$ and gen' can generate the list starting from v_{k+m+1}.



Examples of streams

1. The stream `all-ones` of all ones: `'(1 1 ...)`

```
(define all-ones
  (letrec ([make-ones  (lambda () (cons 1 make-ones))]) (make-ones)))
```

```
> all-ones
'(1 . #<procedure:make-ones>)
```

2. The stream `(nats-from` n`)` of all natural numbers starting from n: `'(`n $n{+}1$ $n{+}2$ $n{+}3$ ...`)`:

```
(define (nats-from n)
  (cons n (lambda () (nats-from (+ n 1)))))
```

```
> (nats-from 7)
'(7 . #<procedure>)
```

Operations on streams

The following operations are designed to work on streams represented in the way described above. First, let's note that, if s is a stream, v is a value and *lst* is a list then `(cons` v s`)` and `(append` *lst* s`)` are streams.

- `(s-take` n s`)` returns the prefix with the first n elements of stream s.

```
(define (s-take n s)
   (cond [(= n 0) null]                    ; base case
         [(procedure? s) (s-take n (s))]   ; unfold s, to get a visible prefix
         [else (cons (car s) (s-take (- n 1) (cdr s)))]))
```

Examples:

```
> (s-take 6 (nats-from 5))
'(5 6 7 8 9 10)
```

```
> (s-take 3 all-ones)
'(1 1 1)
```

- `s-filter` and `s-map` are the generalisations of `filter` and `map` that operate on streams:

```
(define (s-filter p s)
  (cond [(procedure? s) (s-filter p (s))]
        [(p (car s)) (cons (car s) (lambda () (s-filter p (cdr s))))]
        [else (s-filter p (cdr s))]))

(define (s-map f s)
  (cond [(procedure? s) (s-map f (s))]
        [else (cons (f (car s)) (lambda () (s-map f (cdr s))))]))
```

For example, the following are two different ways to define the stream of all natural numbers which are multiple of 3:

```
> (define s3-v1 (s-map (lambda (n) (* 3 n)) (nats-from 0)))
> (define s3-v2 (s-filter (lambda (n) (= (remainder n 3) 0)) (nats-from 0)))
> (s-take 5 s3-v1)      > (s-take 5 s3-v2)
'(0 3 6 9 12)           '(0 3 6 9 12)
```

- `(s-drop` n s`)` returns the stream for s without the first n elements:

```
(define (s-drop n s)
   (cond [(= n 0) s]
         [(procedure? s) (s-drop n (s))]
         [else (s-drop (- n 1) (cdr s))]))
```

- `(s-add` s_1 s_2`)` returns a stream for the list produced by the componentwise addition of elements of s_1 and s_2:

```
(define (s-add s1 s2)
   (cond [(procedure? s1) (s-add (s1) s2)]
         [(procedure? s2) (s-add s1 (s2))]
         [else (cons (+ (car s1) (car s2))
                     (lambda () (s-add (cdr s1) (cdr s2))))]))
```

we can use `s-add` to define the stream `fib` for the list of all Fibonacci numbers `'(`f_1 f_2 f_3 ...`)`, based on the observation that the componentwise addition of `fib` with its tail yields the tail of its tail:

```
'( f1 f2 f3 f4 ... ) +
'( f2 f3 f4 f5 ... )
------------------------
'( f3 f4 f5 f6 ... )
```

Simply put, this means that `fib` =`'(1 1 . lst)` where `lst` is the value of `(s-add (cdr fib) (cddr fib))`. In Racket:

```
(define fib (cons 1 (cons 1 (lambda () (s-add fib (cdr fib))))))
```

For example:

```
> (s-take 12 fib)
'(1 1 2 3 5 8 13 21 34 55 89 144)
```

1.2 Exercises

For this lab work, you can download from Classroom the Racket program `streams.rkt` which contains the previous definitions for streams and operations on streams. Extend this program with the following definitions.

HW1 Suppose s is a stream for the infinite list `'(`v_1 v_1 v_2 `...)`, and m, n are integers such that $0 \leq m \leq n$. Define the operations

(a) `(s-ref` m s`)` which returns the element v_m.

(b) `(s-range` m n s`)` which returns the list if elements `'(`v_m v_{m+1} ... v_n`)`.

HW2 Use `s-add` and `s-map` to define the stream of numbers `as` = `'(a1 a2 a3 ...)` where $a_1 = 1$, $a_2 = 2$ and $a_n = 2\,a_{n-1} - 3\,a_{n-2}$ for all $n > 2$.

HW3 Use `s-map` and `s-filter` to define the stream `primes` of prime numbers: `'( 2 3 5 7 11 13 ...)`. It can be defined by *sieving* the stream of all natural numbers starting from 2. According to Erathostenes, sieving a stream s of numbers yields another stream of numbers s' which is computed as follows:

> If p is the first element of s, then s' starts with p, and is followed by the result of sieving the elements in the tail of s which are not multiple of p.

HW4 Let s_1 =`'(`a_1 a_2 a_3 a_4 `...)` and s_2 =`'(`b_1 b_2 b_3 b_4 `...)` be two streams of numbers in strict increasing order. Define the operations

(a) `(s-merge` s_1 s_2`)`
that produces the stream of elements of s_1 and s_2 in increasing order, without duplicates.
TEST: the first 10 elements of the `merge` of the stream of multiples of 5 with the stream of multiples of 7 is

```
> (s-take 20 (s-merge
         (s-map (lambda (x) (* 5 x)) nats)
         (s-map (lambda (x) (* 7 x)) nats)))
'(0 5 7 10 14 15 20 21 25 28 30 35 40 42 45 49 50 55 56 60)
```

(b) `(sum-stream` s_1 s_2`)` which returns the stream consisting of the sum of elements of streams s_1 and s_2 enumerated in strict increasing order. This means, the resulted stream should enumerate in strict increasing order the elements of the set

$$\{a_i + b_j \mid i, j \in \{1, 2, \ldots\}\},$$

TEST: The first 12 smallest sums of squares of non-zero natural numbers:

```
> (define squares (s-map (lambda (x) (* x x)) (nats-from 1)))
> (s-take 12 (sum-stream squares squares))
'(2 5 8 10 13 17 18 20 25 26 29 32)
```

HW5 The stream `ham` of Hamming numbers is the stream of all numbers of the set

$$H = \{2^m 3^n 5^p \mid m, n, p \in \mathbb{N}\}$$

enumerated in strict increasing order. For example the first 20 elements of this stream are `1 2 3 4 5 6 8 9 10 12 15 16 18 20 24 25 27 30 32 36`

(a) Define `ham`. Observe that `ham` is the stream that starts with 1, followed by the stream produced by merging 3 streams: `ham` multiplied with 2, `ham` multiplied with 3, and `ham` multiplied with 5.

TEST: The first 20 Hamming numbers:

```
> (take 20 ham)
'(1 2 3 4 5 6 8 9 10 12 15 16 18 20 24 25 27 30 32 36)
```

(b) Define the function `(ham-between` m n`)` that returns the number of Hamming numbers in the closed interval $[m..n]$.

```
> (ham-between 20 44)
8
```

because there are 8 Hamming numbers between 20 and 44: 20, 24, 25, 27, 30, 32, 36, 40.

HW6 Define a stream `natPairs` that enumerates all pairs of the set of natural numbers $\mathbb{N} \times \mathbb{N} = \{(a, b) \mid a, b \in \mathbb{N}\}$. In Racket, a pair (a, b) can be encoded as `(cons` a b`)`.

SUGGESTION. First, define `(pairsFor n)` which computes the list if all pairs $(a, b) \in \mathbb{N} \times \mathbb{N}$ with $a+b =$ `n`. Next, define the function `(pairsFrom n)` which computes a stream of all $(a, b) \in \mathbb{N} \times \mathbb{N}$ with $a + b \geq$ `n`. `pairsFrom` is easy to define it you observe the relation between `(pairsFrom n)` and `(pairsFrom (+ n 1))`. Then `natPairs` is the stream computed by `(pairsFrom 0)`.

HW7 (Bonus) Let `a` = `'(`a_0 a_1 a_2 ...`)` and `b` = `'(`b_0 b_1 b_2 ...`)` be two streams of numbers. Define `(prod-stream a b)` which computes the stream `c` = `'(`c_0 c_1 c_2 ...`)` where

$$c_i = \sum_{j=0}^{i} a_j \cdot b_{i-j}$$

for all $i \in \mathbb{N}$.

SUGGESTION: `a` and `b` are the streams of coefficients of $x^0, x^1, x^2, \ldots$ in the infinite polynomials

$$\texttt{pa} = a_0 + a_1\,x + a_2 x^2 + a_3 x^3 + \ldots \qquad \texttt{pb} = b_0 + b_1\,x + b_2 x^2 + b_3 x^3 + \ldots$$

Then `c` is the stream of coefficients of $x^0, x^1, x^2, \ldots$ in the polynomial `pa` $\cdot$ `pb`. To identify a definition for the stream `c`, use the fact that

$$\texttt{pa} \cdot \texttt{pb} = (a_0 + \texttt{pa}_1 \cdot x)(b_0 + \texttt{pb}_1 \cdot x) = a_0 \cdot b_0 + b_0 \cdot \texttt{pa}_1 \cdot x + a_0 \cdot \texttt{pb}_1 \cdot x + \texttt{pa}_1 \cdot \texttt{pb}_1 \cdot x^2$$

where $\texttt{pa}_1 = a_1 + a_2\,x + a_3 x^2 + a_4 x^3 + \ldots$

$\texttt{pb}_1 = b_1 + b_2\,x + b_3 x^2 + b_4 x^3 + \ldots$

TEST:

```
> (s-take 10 (prod-stream all-primes all-primes))
'(4 12 29 58 111 188 305 462 679 968)
> (s-take 10 (prod-stream fib fib))
'(1 1 2 3 5 8 13 21 34 55)
> (s-take 10 (prod-stream all-primes fib))
'(2 5 12 24 47 84 148 251 422 702)
```